2023 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit) | 979-8-3503-1102-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/EUCNC/6GSUMMIT58263.2023.10188344

DESK: Distributed Observability Framework for Edge-Based Containerized Microservices

Muhammad Usman*, Simone Ferlin*†, Anna Brunstrom*, Javid Taheri*
*Department of Mathematics and Computer Science, Karlstad University, Karlstad, Sweden
†Red Hat, Stockholm, Sweden
Email: *muhammad.usman@kau.se, *†sferlinr@redhat.com, *anna.brunstrom@kau.se, *javid.taheri@kau.se

***Abstract*—Modern information technology (IT) infrastructures are becoming more complex to meet the diverse demands of emerging technology paradigms such as 5G/6G networks, edge, and internet of things (IoT). The intricacy of these infrastructures grows further when hosting containerized workloads as microservices, resulting in the challenge to detect and troubleshoot performance issues, incidents or even outages of critical use cases like industrial automation processes. Thus, fine-grained measurements and associated visualization are essential for operation observability of these IT infrastructures. However, most existing observability tools operate independently without systematically covering the entire data workflow. This paper presents an integrated design for multi-stage observability workflows, denoted as *DistributEd obServability frameworK (DESK)*. The proposed framework aims to improve observability workflows for measurement, collection, fusion, storage, visualization, and notification. As a proof of concept, we deployed the framework in a Kubernetes-based testbed to demonstrate the successful integration of various components and usability of collected observability data. We also conducted a comprehensive study to determine the caused overhead by DESK agents at the reasonably powerful edge node hardware, which shows on average a CPU and memory overhead of around 2.5% of total available hardware resource.**

***Index Terms*—Edge Computing, Internet of Things (IoT), Microservices, Monitoring, Observability, 5G/6G.**

I. Introduction

With help of the 5^{th} generation of mobile networks (5G), the industry is entering the digitization era, where smart and well-connected communication and processing chains are demanding new requirements. As an example, realizing factory automation, managing industrial processes and devices, necessitates strict real-time requirements, including the 5G network edge infrastructure itself. To date, this infrastructure is mainly realized via lightweight containerization technologies (e.g., *Docker*) and managed by container orchestration platforms (e.g., *Kubernetes*) to provide flexibility and scalability.

When industrial applications previously run on dedicated hardware are transformed into a collection of containerized microservices at the edge of a 5G network [1], meeting their required real-time guarantees becomes very challenging. In this context, fine-grained monitoring[1] is a critical requirement to understand performance degradation as well as detect incidents such as service violations or even outages.

[1]Monitoring is a method of gaining visibility into different components of a system to diagnose different types of operational anomalies [2].

Even though monitoring has served as a core function of IT for the last few decades, it started to prove itself incomplete due to various factors and practices including agile development, cloud deployments, and DevOps. These factors have changed how the entire IT environment (e.g., infrastructures, platforms, and applications) should be observed to promptly respond to incidents. This transformation proved itself powerful to be adopted by mobile networks such as 5G, which is considered a critical infrastructure and, as such, it is also expected to accommodate critical use case requirements. To this end, observability is an emerging set of practices and tools that goes beyond monitoring with the goal to show *why* a system is not operative, thus providing insights of internal state of a system by analyzing its external outputs [3]. The core idea in observability is to quickly learn what is happening in the system by quickly identifying the root cause of the problem. Commonly, three main data types (logs, metrics, and traces) are used by an observability system. Thus, a more future-looking observability framework for real-time performance observability of container-based edge services must capture performance of the various infrastructure components as well as the performance of the applications leveraging these three data types, along with methods for correlating the symptoms found in the collected data.

While there are extensive studies on monitoring [4]–[9], a comprehensive observability framework that can facilitate flexible processing and integration of observability data from multiple parts of networks, IT infrastructure and applications remains a gap. Thus, in this paper, we propose an integrated design for a distributed observability workflow, denoted as *DistributEd obServability frameworK (DESK)*, by uniquely integrating various observability-related open-source upstream software. Specifically, we propose a practical framework design for an entire data processing workflow, from measurement to visualization. Also, we investigate related open-source tools and platforms that are employed in the software implementation of DESK. Furthermore, we perform verification of the framework's prototype implementation in a locally configured small testbed.

The rest of this paper is structured as follows. Section II briefly discusses related works. Section III presents the design and implementation details of DESK, followed by verification results and some discussion in Section IV. Finally, Section V concludes the paper.

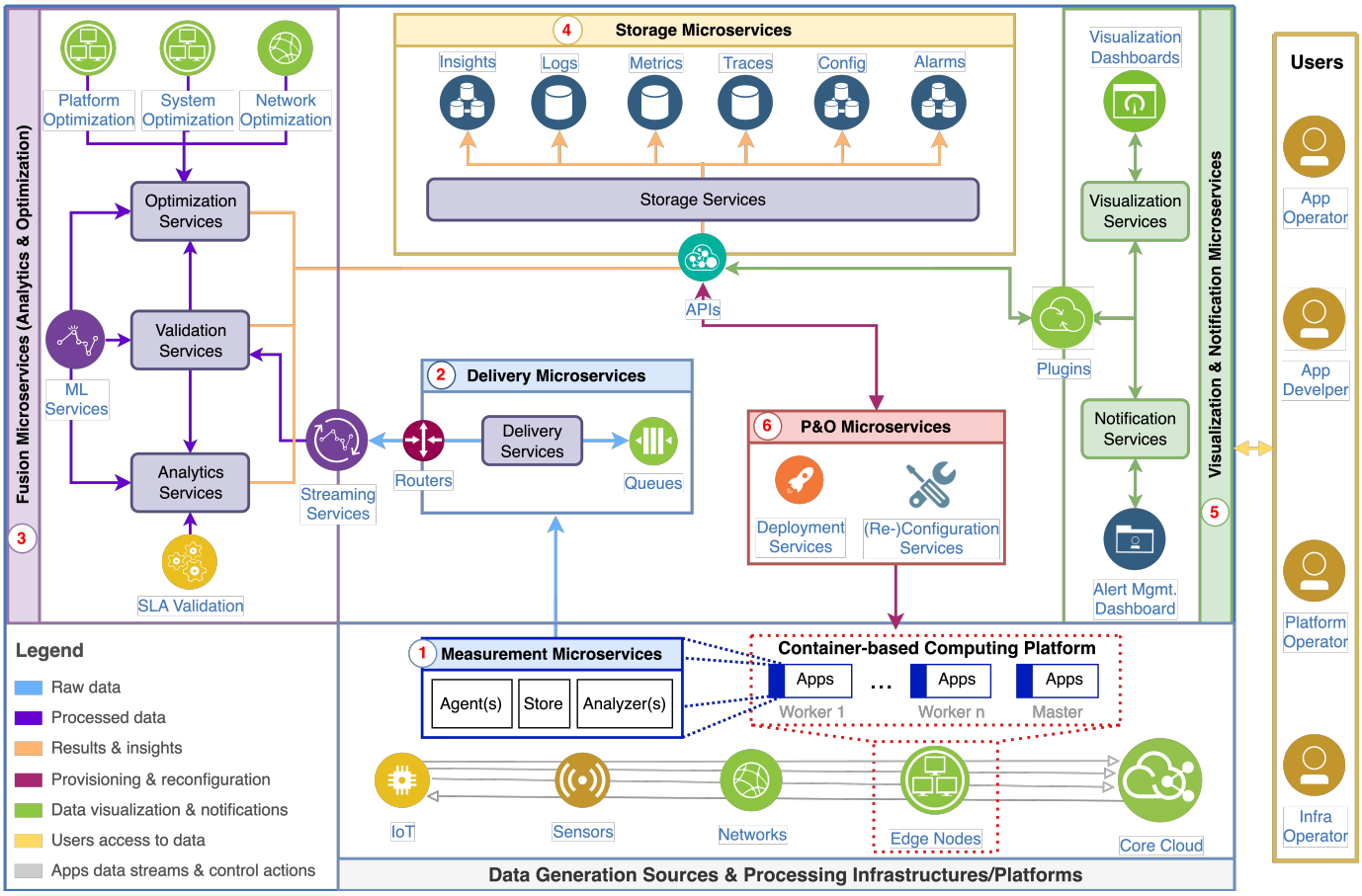


Fig. 1: A detailed architecture of DESK for collecting, processing, and analyzing observability data from multiple sources.

II. Related Work

There are numerous studies on monitoring that partly cover observability-level details. Authors in [4] proposed a framework for coordinating monitoring agents and their back-ends to allow monitoring workflows at edge nodes for gathering system-level performance metrics. [10] proposed a unified monitoring framework for prompt identification of operational faults in distributed clouds. [6] proposed a bottom-up NFV analysis platform to systematically profile and classify Virtual Network Functions (VNFs) considering different requirements. [7] proposed an eBPF-based microservices collection scheme that delivers diverse system metrics for single tenant environment. [11] discussed techniques to improve the observability of distributed applications by combining multiple data sources to provide a comprehensive view of the application's operation with the aim of better decision-making in the orchestration process. From open-source community, OpenTelemetry [12] is a project for collecting observability data from cloud-native applications and their infrastructure to observe overall health and performance. This project aims to standardize the generation and collection of telemetry data in distributed systems. Aside from academic works and open-source observability solutions, several proprietary solutions are also available in [9]. In summary, although there are existing works in this domain, we note a gap in the literature around an observability solution that can enable low overhead measurements and flexible processing of them to aid in the smooth operation of edge-based containerized microservices.

III. DESK: Design and Implementation

We are designing DESK to deal with metrics, logs, and traces together with fine-grained visualization support to assist a diverse group of operators (e.g., infrastructure, platform, and application) in efficiently managing and controlling an edge-based system. More precisely, it targets to support end-to-end runtime performance observability with infrastructure, platform, and containerized microservices for IoT applications in the Kubernetes-orchestrated edge cluster. As depicted in Fig. 1, DESK consists of six main stages. These stages are made of smaller, more manageable components in order to provide loosely coupled data workflows. To isolate these components from impacting actual workloads, they are deployed separately using two dedicated namespaces (i.e., tenants) with limited resource allocations. One namespace is termed as *measurement* and it contains the measurement stage components. Other namespace is termed as *observability*, and it contains all the other components of DESK. The framework

implementation is kept open-source and available at GitHub[2]. Next, we describe the main stages of DESK.

1) Measurement Stage: Fine-grained metrics from heterogeneous measurement points (MPs) are essential to permit real-time observability tools to work with a large variety of data during analysis. Nevertheless, selecting the appropriate metrics and measurement intervals for obtaining operation observability is challenging. Generally, it is determined based on factors such as the number of active users, the periodicity of edge node aliveness shifts, and the impact of edge node failures on the running applications [13]. Considering these requirements, DESK employs *Telegraf*, *Promtail*, and *Jaeger* agents due to their lightweight nature and data enrichment features. To acquire metrics, we employ Telegraf agent. Logs data is acquired using Promtail agent. For traces acquisition, Jaeger agent is selected. Although DESK does not prioritize agents for other sources, such as underlying mobile networks, it does support observability data ingestion from such sources in a push mode. Furthermore, automated deployment of these agents on newly added edge nodes and their persistent execution are ensured using the Kubernetes Daemonset object, which guarantees one running agent per node.

2) Delivery Stage: For data delivery and collection, reliable data transfer into a central location is required, where majority of the services are running. In fact, moving data among different services has several steps such as copying and integrating data with other data sources. Therefore, a data pipeline integrates all of these steps to ensure consistency. Since a Kubernetes-based edge cluster consists of multiple worker nodes, a fault-tolerant collection and aggregation of data in near real-time from distributed edge nodes to produce centralized operational data feeds is a critical requirement. For reliable data transfer from edge nodes to centralized location, the DESK data pipeline uses *Apache Kafka*, which is considered an industry standard with high-performance and reliability. It provides a well-defined set of APIs for various language platforms, and the DESK implementation leverages JAVA-based APIs.

3) Fusion Stage: Data fusion in the context of DESK is the process of observing incoming data to draw conclusions about the overall systems and applications' performance, primarily with the assistance of reliable data cleaning and efficient data analysis systems. The data under observation could be historical or real-time, depending on the requirements of the troubleshooting or diagnostics procedures. Most of the effort required, especially for these analytics and optimization procedures, is spent on preparing and integrating data to produce accurate results. Since this stage is highly dependent on an application use case; therefore, we have not yet focused heavily on implementing the components of this stage.

4) Storage Stage: To deliver a data-centric framework, flexible storage and fast data access are two critical factors. Data storage is a challenging operation because of numerous data types and formats (e.g., metrics, logs, traces). Since DESK is a data-centric framework, it carefully weighs data storage by delivering elastic storage for manageable maintenance (i.e., via proper data organization for long-term and short-term storage), as well as for performance and scalability (i.e., via careful data split into various datastores for metrics, logs, and traces). DESK applies *Prometheus* for data storage of metrics. For logs, the framework employs *Loki* [14], which is a horizontally scalable, multi-tenant log aggregation system motivated by Prometheus. Furthermore, *Elasticsearch* is used for traces storage and long-term storage of the data.

5) Visualization and Notification Stage: Visualization is a critical tool for the operators who need to take immediate recovery actions after viewing data in real-time. In particular, it helps detecting patterns visually and taking measures against undesirable operational behaviors. DESK applies *Grafana* open-source software for creating visualizations and dashboards. Grafana natively supports over thirty data sources, including Loki, Prometheus, and Elasticsearch, which are the backend datastores of DESK.

Alerting is a reactive component of the framework that initiates actions when metric values change. DESK includes anomaly detection-based alerting and heartbeat alerts in addition to threshold-based alerts. It transmits alerts to the *Prometheus AlertManager*, which handles them through a pipeline of silencing, inhibition, grouping, and email notification or quicker actions for simplified operational issues via P&O stage automated (re-)configuration components.

6) Provisioning and Orchestration (P&O) Stage: DESK P&O services are divided into two main categories. First, deployment services are responsible for installing and running a specific number of DESK components/services on the edge nodes. DESK itself is a fully-containerized service and fully-manageable with Kubernetes-native features. Mainly, we utilized *Helm* [15] for installing required framework services in a scalable manner using JSON-based configuration files. Second, (re-)configuration services are in charge of taking over generated actions, and invoking deployed microservices and system re-configurations when necessary using *Kubectl* tool.

IV. DESK: Verification Results and Discussion

This Section presents DESK's experimental verification results. A thorough investigation is carried out in Section IV-B1 using verification setup, as described in Section IV-A, to determine how much overhead observability measurements incur. Also, an analysis of how collected measurements can help different user groups to observe system performance using temporal data is provided in Section IV-B2.

A. Experimentation Environment

An edge infrastructure can be build using different resource specifications. For instance, an enterprise can set up few powerful servers or several *Raspberry Pis* according to the the business use case demands. Our testbed configuration took an intermediate approach, where desktop-based hardware with average specifications are employed. These hardware are setup as a small cluster and it is driven by Kubernetes for

[2]https://github.com/smartx-usman/distributed-observability-framework.git

DESK deployment and analysis. This testbed setup includes one master node and three worker nodes. All these nodes have an Intel(R) Core(TM) i7-6700 CPU running at 3.40GHz, 16 GB of DDR4 memory, 256 GB of disk space, a 1 Gbit/s network interface, Ubuntu 22.04 operating system (OS), and the Kubernetes software version 1.26.0 installed. Furthermore, we did not investigate the impact of edge node's geographical placement and inter-connectivity issues during these experiments; rather, we assumed that all edge nodes are dependable and stably inter-connected.

ThingsBoard[3] is an open-source edge IoT platform; we deployed it for application data processing and load generation in our testbed. Other than ThingsBoard microservices, a custom-developed IoT data generation application is also deployed in the testbed as a set of pods (i.e., deployment) to simulate a continuous stream of sensor data. This application generates each message with four data points, and transports the simulated data to the ThingsBoard.

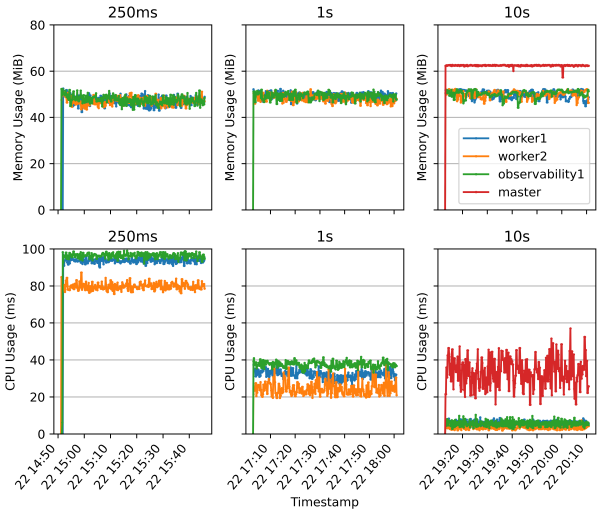
B. Experimentation Results

1) Measurements Overhead Results: An observability measurement agent introduces a certain overhead, because it uses the system resources (e.g., CPU) for performing its tasks. These measurement agents can be a source of contention, particularly in constrained edge-based infrastructures where hundreds of performance counters are collected and stored. Because DESK aims to gather various types of data, it is critical to ensure it does not generate excessive overhead. Hence, the impact of agent overhead is thoroughly investigated.

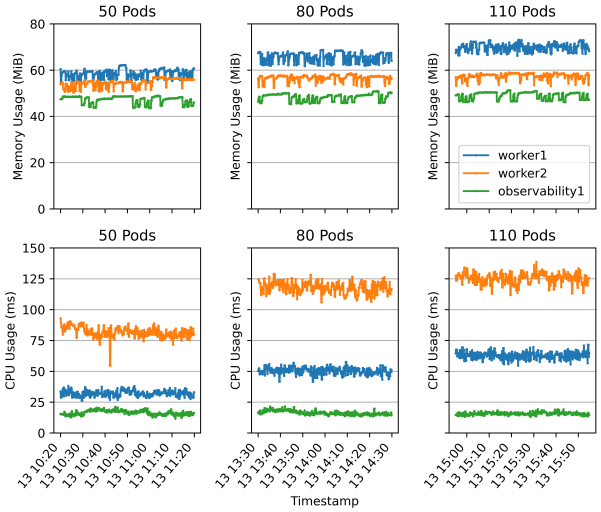
DESK supports metrics collection, and the overhead caused by these measurements is depicted in Fig. 2a. Using eight input plugins, a total of 88 metrics are first gathered by Telegraf and then sent via output plugin for further processing. To observe metrics overhead, various configurations, such as measurement intervals between 250ms and 10s, are tested.

An identical memory usage is recorded throughout all these varying intervals, and it remained consistent at around 50 MiB. However, the memory overhead touched 62 MiB for the master node case of 10s (please note, lower than 10s intervals were not successful on this node because this agent was configured with an upper resource constraint of 80 MiB). This difference is due to nine input plugins on the master node as opposed to eight input plugins on the worker nodes. The additional input plugin on the master node generated Kubernetes resource metrics that necessitated more resources to process due to their high cardinality. Thus, we recommend a less aggressive measurement interval for these metrics on the master node (e.g., 10s), because a more aggressive interval will only cause errors and higher overhead. Compared to memory usage, a more significant difference is noted in CPU usage when we reduced the measurement interval from 250ms to 10s. Please note that Kubernetes default monitoring interval is set to 10s; whereas, we consider a measurement interval of 1s as a better choice for time-sensitive applications. A lower measurement

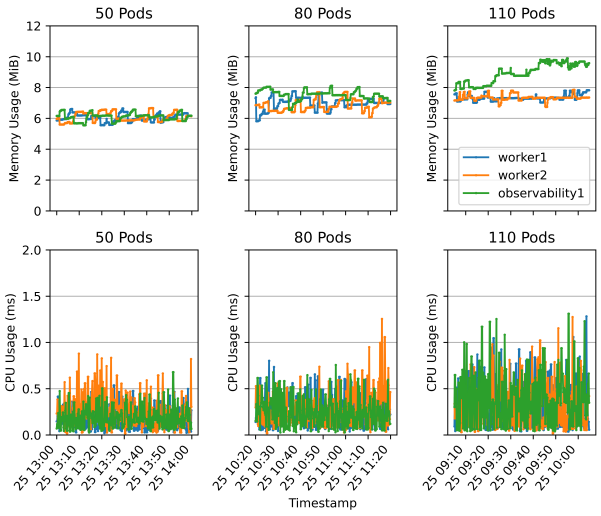
[3]https://thingsboard.io/



(a) Metrics measurements overhead.



(b) Logs measurements overhead.



(c) Traces measurements overhead.

Fig. 2: Observability data measurement and collection overhead incurred by DESK deployment on edge nodes.

interval will allow faster detection, and this will ultimately allow Kubernetes to schedule/restart a new container without waiting for 10s period. Furthermore, DESK allows to specify a different measurement interval for each group of metrics. For example, the measurement interval for CPU consumption metrics can be set to 1s, whereas the measurement interval for Kubernetes resource metrics can be set to 10s.

DESK supports logs collection, and the overhead caused by logging agent is depicted in Fig. 2b. These experiments were performed with a varying numbers of pods. At *Google Cloud Engine*, 100 are the maximum number of recommended pods per worker node[4] while Kubernetes official documentation recommends a maximum of 110 pods per worker node[5]; therefore, we experimented with a maximum of 110 pods. The logs measurement and metadata tags augmentation overhead went up to 70 MiB for 110 pods on one of the worker nodes. Compared to memory overhead, CPU overhead varied more and reached 90ms for 50 pods and 130ms for 110 pods.

DESK supports trace collection, and the overhead caused by these trace measurements is presented in Fig. 2c. For trace generation, we instrumented a Python application with OpenTelemetry tracing SDKs and APIs. The generated traces are then sent from this Python application to the Jaeger agent that was deployed on each worker node and load-balanced with the help of a Kubernetes service object. Simulated sensor data from 50, 80, and 110 pods were continuously sent to the ThingsBoard platform and then processed by our Python application, which generated traces.

For traces, memory overhead remained around 6.5 MiB with 50 pods and increased to 10 MiB with 110 pods. *Observability1* worker node behaved slightly differently than other worker nodes, possibly due to internal resource rescheduling by Kubernetes. For 110 pods, CPU resource overhead reached up to 1.3ms. To summarize, we find that the key consideration with tracing-enabled applications is determining an appropriate sampling strategy to ensure a sustainable and actionable load for the tracing backend services.

2) Observability Data Usage Results: The real value of collected metrics, logs, and traces can only be realized if they can reveal any potential operational flaws, particularly for different user groups. To conduct a usability study of DESK, we generated a continuous stream of simulated sensor data with the assumption that all the sensors generate the same amount of data and data is sent to the ThingsBoard for further processing. For each simulated sensor, we defined three states (i.e., *normal*, *abnormal*, and *mixed*). In a normal sensor state, valid data values are generated and transmitted at regular intervals. When a sensor is in abnormal state, valid data values are generated but transmitted irregularly. In a mixed sensor state, both valid and invalid data values are generated and transmitted irregularly. As shown in Fig. 3, the pods processing data from the *normal* sensors generated almost 600 logs/min, whereas CPU usage was kept around 8ms. The pod labeled as *abnormal* generated around 200 logs/min and had a CPU usage of 2ms. These figures are significantly lower than normal sensor pods and exhibit some irregularity. *Mixed* pod generated 4500 logs/min and consumed nearly 45ms of CPU time. These figures are significantly higher when compared to normal sensor pods, indicating another abnormality. Thus, from the platform operator's perspective, without knowing any application-specific details, an operational fault alarm can be triggered for further troubleshooting, while the application operator can investigate further based on the log contents.

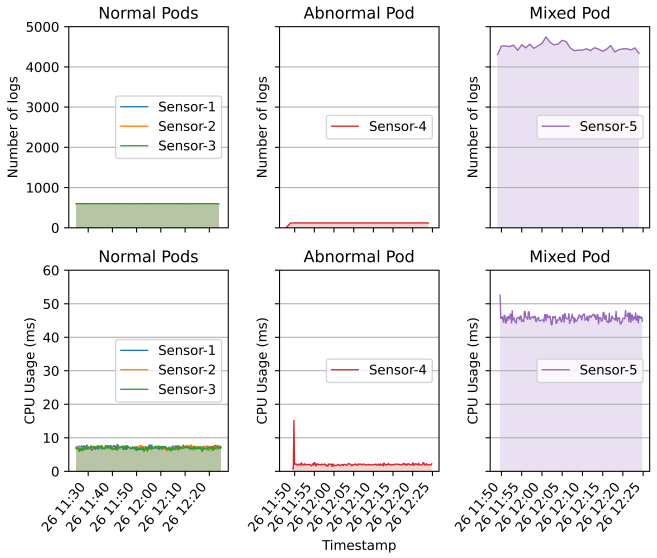
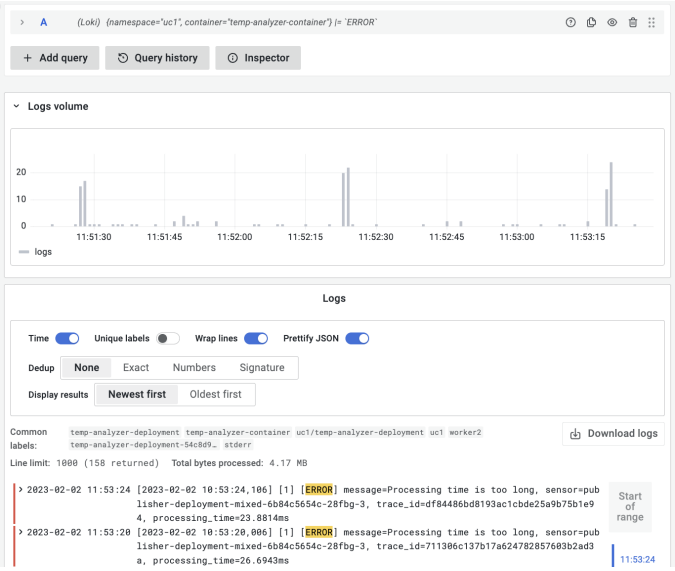


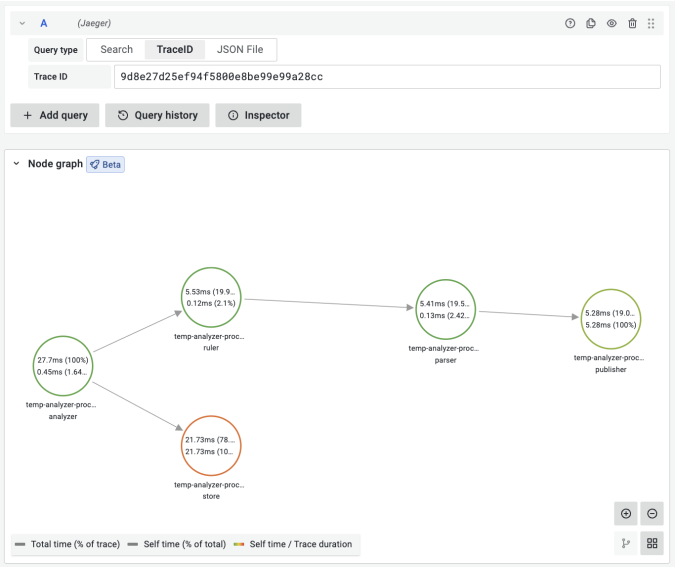
Fig. 3: Logs and metrics analysis to detect any faulty pods.

When observing end-to-end application performance, integrated analysis and exploration of observability data is a primary requirement. For this purpose, logs and traces integrated analysis is conducted, and some results are shown in Fig. 4. The same setup of ThingsBoard with a microservices-based Python application is utilized during this experimentation. To enable integration and exploration of traces from the logs, a *traceid* is incorporated programmatically in logs of particular interest. Then, using the Grafana visualization, a regular expression is created to automatically establish a navigable link to the trace for further exploration (e.g., spans). For example, in the case presented in Fig. 4a, whenever the application processing delay exceeds 20ms, an error log is generated with the corresponding traceid, incorporated as a tag. When application operators get the filtered observability data in the form of visualization, they can select a specific log to obtain detailed information from the trace as well as the time spent by each microservice while processing the data. If we look at Fig. 4b, the main reason for a delay greater than 20ms is caused by the *temp-analyzer-proc-store* service, which is in charge of storing data in a MySQL database. Based on this information, different operations teams (e.g., platform operators) can perform additional troubleshooting to resolve storage delay issues.

[4]https://learnk8s.io/kubernetes-node-size
[5]https://kubernetes.io/docs/setup/best-practices/cluster-large/



(a) Logs filtering and visualization.



(b) Traces filtering and visualization.

Fig. 4: Logs and traces integrated analysis using visualizations to detect any operational faults.

V. Conclusion

In this paper, we presented the DESK architecture and experimental results. Compared to previous works, DESK's main advantage is its loosely coupled integrated design, which provides flexibility when processing various types of observability data, allowing operators to troubleshoot and resolve complex operational issues quickly. Specifically, DESK offers an integrated design of an observability workflow for edge-based containerized microservices, and its operation has been thoroughly verified in terms of measurements processing overheads. For our future work, we aim to further develop and integrate various components in DESK, including the validation with other use cases. More precisely, DESK requires rigorous testing in a dedicated testbed, which is designed to run high-volume 5G applications at the edge, to verify that all components are operational in high workload scenarios, and it aids in improving the operations of the environment.

Acknowledgment

This work was supported by the Knowledge Foundation of Sweden (KKS) through the Synergy Project AIDA - A Holistic AI-driven Networking and Processing Framework for Industrial IoT (Rek:20200067).

References

[1] S. Baškarada, V. Nguyen, and A. Koronios, "Architecting Microservices: Practical Opportunities and Challenges," *J. Comput. Inf. Syst.*, vol. 60, no. 5, pp. 428–436, Sep. 2020, publisher: Taylor & Francis.

[2] O. Korableva, O. Kalimullina, and E. Kurbanova, "Building the Monitoring Systems for Complex Distributed Systems: Problems and Solutions:," in *Proc. 19th Int. Conf. on Enterprise Inf. Syst.*, Porto, Portugal, 2017, pp. 221–228.

[3] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A Survey on Observability of Distributed Edge & Container-Based Microservices," *IEEE Access*, vol. 10, pp. 86 904–86 919, 2022.

[4] A. Brandón, M. S. Pérez, J. Montes, and A. Sanchez, "FMonE: A Flexible Monitoring Solution at the Edge," *Wirel Commun Mob Comput*, vol. 2018, p. e2068278, Nov. 2018, publisher: Hindawi.

[5] M. Usman and J. Kim, "SmartX Multi-View Visibility Framework for unified monitoring of SDN-enabled multisite clouds," *Trans Emerging Tel Tech*, vol. 33, no. 8, p. e3819, 2022.

[6] M. Gokan Khan, S. Bastani, J. Taheri, A. Kassler, and S. Deng, "NFV-Inspector: A Systematic Approach to Profile and Analyze Virtual Network Functions," in *Proc. IEEE 7th Int. Conf. on Cloud Netw. (CloudNet)*, Oct. 2018, pp. 1–7.

[7] J. Levin and T. A. Benson, "ViperProbe: Rethinking Microservice Observability with eBPF," in *Proc. IEEE 9th Int. Conf. on Cloud Netw. (CloudNet)*, Nov. 2020, pp. 1–8.

[8] C. Lai, J. Kimball, T. Zhu, Q. Wang, and C. Pu, "milliScope: A Fine-Grained Monitoring Framework for Performance Debugging of n-Tier Web Services," in *Proc. IEEE 37th Int. Conf. on Distrib. Comput. Syst. (ICDCS)*, Jun. 2017, pp. 92–102.

[9] A. Thurai, "GigaOm Radar for Cloud Observability," Feb. 2021, last Modified: 2021-10-27 Publisher: Gigaom. [Online]. Available: https://gigaom.com/report/gigaom-radar-for-cloud-observability/

[10] M. A. Rathore, M. Usman, and J. Kim, "Maintaining SmartX multi-view visibility for OF@TEIN+ distributed cloud-native edge boxes," *Trans Emerging Tel Tech*, vol. 32, no. 6, p. e4101, 2021.

[11] I. Tzanettis, C.-M. Androna, A. Zafeiropoulos, E. Fotopoulou, and S. Papavassiliou, "Data Fusion of Observability Signals for Assisting Orchestration of Distributed Applications," *Sensors*, vol. 22, no. 5, p. 2061, Jan. 2022.

[12] "OpenTelemetry: Documentation." [Online]. Available: https://opentelemetry.io/docs/

[13] M. Usman, A. C. Risdianto, J. Han, and J. Kim, "Interactive Visualization of SDN-Enabled Multisite Cloud Playgrounds Leveraging SmartX MultiView Visibility Framework," *Comput. J.*, vol. 62, no. 6, pp. 838–854, Jun. 2019.

[14] E. Bautista, N. Sukhija, and S. Deng, "Shasta Log Aggregation, Monitoring and Alerting in HPC Environments with Grafana Loki and ServiceNow," in *Proc. IEEE Int. Conf. on Cluster Comput. (CLUSTER)*, Sep. 2022, pp. 602–610.

[15] S. Gokhale, R. Poosarla, S. Tikar, S. Gunjawate, A. Hajare, S. Deshpande, S. Gupta, and K. Karve, "Creating Helm Charts to ease deployment of Enterprise Application and its related Services in Kubernetes," in *Proc. Int. Conf. on Comput., Commun. and Green Eng. (CCGE)*, Sep. 2021, pp. 1–5.